

Dynamic Programming

FAANG Interview

Revision Guide

Knapsack | LCS | MCM | Standalone DP

Tricks, Patterns, Templates & Questions

Complete guide with Recursion, Memoization & Bottom-Up for every problem

From basic identification to advanced FAANG-level optimizations

Table of Contents

Chapter 1: The DP Masterplan

How to identify & solve ANY DP problem

Chapter 2: 0/1 Knapsack Family

7 problems from one template

Chapter 3: Unbounded Knapsack Family

10 problems including Rod Cut, Coin Change, Word Break

Chapter 4: LCS Family

16 problems including Edit Distance, Palindromes

Chapter 5: MCM / Partition DP Family

8 problems including Burst Balloons, Egg Drop

Chapter 6: Standalone DP Problems

Count Ways, Jump Game, House Robber, Knight Dialer, etc.

Chapter 7: Ultimate Cheat Sheet

All formulas, patterns & tricks on one page

Chapter 8: Visual Architecture Diagrams

Family trees, conversion flows, table filling visuals

Chapter 1: The DP Masterplan

How to identify and solve ANY Dynamic Programming problem in interviews

The 4 DP Families (Learn 4 templates, solve 40+ problems)

Family	# Problems	Key Idea	Identification
0/1 Knapsack	7	Pick or Skip	Choice + Capacity
Unbounded KS	10	Reuse items	Infinite supply
LCS	16	Two string compare	Common/Convert
MCM	8	Split range at k	Partition range

The 3-Step Recipe (works for EVERY DP problem)

Step 1: Write RECURSION with base case + choice diagram

Think: 'What choices do I have at each step?' Write the brute force.

Step 2: Add MEMOIZATION (cache changing variables)

Find variables that change in recursion. Create dp table. Add 3 lines: check cache, compute, store.

Step 3: Convert to BOTTOM-UP (tabulation)

Replace recursion with loops. Base case becomes initialization. Recursive calls become table lookups.

How to Identify DP in 5 Seconds

"maximize/minimize" + "choices at each step" --> DP
 "count number of ways" --> DP
 "can you reach / is it possible" --> DP
 "optimal substructure" + "overlapping subproblems" --> DP
 Greedy FAILS (need to try all combinations) --> DP

Recursion to Memoization (3 Lines Added)

```
# Line 1: Create memo table
memo = {} # or memo[n+1][w+1] = -1

# Line 2: Check before computing
if key in memo: return memo[key]

# Line 3: Store before returning
memo[key] = result
return memo[key]
```

Memoization to Bottom-Up Conversion

TRICK: Base case in recursion = Initialization in Bottom-Up

if $n==0$: return 0 --> $dp[0][j] = 0$ for all j

Recursive call $f(n-1, w)$ --> $dp[i-1][j]$ in table

The changing variables (n, w) become loop dimensions (i, j)

Chapter 2: 0/1 Knapsack Family

7 problems from ONE template - Learn once, solve all

Identification: How to spot Knapsack in 5 seconds

Question 1: "Am I making a PICK or SKIP choice for each item?" --> YES

Question 2: "Is there a CONSTRAINT (capacity, sum, limit)?" --> YES

BOTH YES = Knapsack Family!

"Pick or Skip" + "Constraint" = KNAPSACK

The Choice Diagram (draw this in EVERY interview)

```

item i (weight = wt[i], value = val[i])
      /           \
    wt[i] <= W?   wt[i] > W?
    (CAN pick)   (TOO heavy)
      /       \           |
    PICK      SKIP      SKIP
val[i] +      solve(W,      solve(W, n-1)
solve(W -    n-1)
wt[i], n-1)

Answer = max(PICK, SKIP) if can pick
Answer = SKIP           if too heavy
  
```

The Universal Template

```

def knapsack(wt, val, w, n):
    if n == 0 or w == 0:
        return 0
    if wt[n-1] <= w:
        pick = val[n-1] + knapsack(wt, val, w - wt[n-1], n-1)
        skip = knapsack(wt, val, w, n-1)
        return max(pick, skip)
    else:
        return knapsack(wt, val, w, n-1)
  
```

Bottom-Up Template

```

dp[i][j] = 0 for i=0 or j=0 (base case)

for i in range(1, n+1):
    for j in range(1, W+1):
        if wt[i-1] <= j:
            dp[i][j] = max(val[i-1] + dp[i-1][j - wt[i-1]], # pick
                           dp[i-1][j])                      # skip
  
```

```

else:
    dp[i][j] = dp[i-1][j] # must skip

```

The 7 Knapsack Children (all from same template)

Problem	dp type	Operator	Target (W)	LC #	Key Change
0/1 Knapsack	int	max()	capacity	-	Base problem
Subset Sum	bool	OR	target S	-	Can we reach sum?
Equal Partition	bool	OR	sum/2	416	totalSum must be even
Count Subsets	int	+	target S	-	How many subsets?
Min Subset Diff	bool	OR	all sums	1049	min(total - 2*S1)
Target Sum	int	+	(T+tgt)/2	494	+/- assignment
#Subsets Diff	int	+	(T+diff)/2	-	S1 - S2 = diff

TRICK: 3 types of operators

"Can we?" (yes/no) --> bool, use OR (Subset Sum, Partition)
 "How many?" (count) --> int, use + (Count Subsets, Target Sum)
 "Best value?" (optimize) --> int, use max() (Knapsack)

Space Optimization: $O(n*W)$ to $O(W)$

TRICK: 1D Array - RIGHT to LEFT

```

for i in range(n):
    for j in range(W, wt[i]-1, -1): # RIGHT to LEFT!
        dp[j] = max(dp[j], val[i] + dp[j - wt[i]])

```

WHY right to left? Left to right would use updated values = using item TWICE!

Chapter 3: Unbounded Knapsack Family

10 problems - 'Infinite supply' = stay on same row

The ONE-CHARACTER Difference from 0/1 Knapsack

0/1: $dp[i][j] = \max(val[i-1] + dp[i-1][j-wt[i-1]], dp[i-1][j])$

Unbounded: $dp[i][j] = \max(val[i-1] + dp[i][j-wt[i-1]], dp[i-1][j])$

TRICK: STAY vs GO

0/1 Knapsack: GO to previous row (i-1) -- item used ONCE, move on

Unbounded: STAY on same row (i) -- item can be REUSED!

Memory: 'Unbounded = Unlimited = U stay on same row'

Identification: How to know it's UNBOUNDED?

"infinite supply" --> UNBOUNDED

"unlimited quantity" --> UNBOUNDED

"can use multiple times" --> UNBOUNDED

"at most once" --> 0/1 Knapsack

1D Space Optimization: LEFT to RIGHT

TRICK: Direction matters!

0/1 Knapsack 1D: j from RIGHT to LEFT (W down to wt[i])

Unbounded 1D: j from LEFT to RIGHT (wt[i] up to W)

LEFT-to-RIGHT uses updated values = reusing same item (unbounded!)

RIGHT-to-LEFT uses old values = item used only once (0/1)

The 10 Unbounded Problems

Problem	items	values	capacity	Optimize	LC #	Key Trick
Rod Cutting	length[]	price[]	rod_len	MAX	-	lengths are weights
Coin Change I	coins[]	1	target	COUNT	518	Add instead of max
Coin Change II	coins[]	1	target	MIN	322	Init with INF
Integer Break	[1..n-1]	product	n	MAX	343	Multiply not add
Perfect Sq.	[1,4,9..]	1	n	MIN	279	Coins = squares

Min Cost Tix	[1,7,30]	costs	maxday	MIN	983	Skip non-travel
Word Break	words	bool	len(s)	OR	139	Substring match
Word Break Cnt	words	1	len(s)	COUNT	IK	dp[i] += dp[j]
Combo Sum IV	nums[]	1	target	COUNT	377	Swap loops!

TRICK: Combinations vs Permutations (swap the loops!)

Combinations (order doesn't matter): items outer, target inner

Permutations (order matters): target outer, items inner

Coin Change Count = combinations (items outer)

Combination Sum IV = permutations (target outer)

Universal 1D Template (covers 90% of problems)

```
def solve(items, target):
    dp = [BASE] * (target + 1)    # INF for MIN, 0 for MAX, 0 for COUNT
    dp[0] = INIT                 # 0 for MIN/MAX, 1 for COUNT

    for item in items:           # outer: items
        for j in range(item, target + 1):  # inner: LEFT to RIGHT
            dp[j] = OP(dp[j], COMBINE(dp[j - item]))

    return dp[target]
```


Chapter 4: LCS (Longest Common Subsequence) Family

16 problems - the MOTHER of all string DP

Identification: How to spot LCS problems

TWO strings (or string vs its reverse) --> LCS family
 "subsequence" --> LCS (order matters, not contiguous)
 "substring" --> LCS with RESET on mismatch
 "palindrome" --> LCS(s, reverse(s))
 "convert A to B" --> LCS to find common, rest = operations

The Choice Diagram

Compare $X[i-1]$ vs $Y[j-1]$:

MATCH ($X[i-1] == Y[j-1]$): $1 + dp[i-1][j-1]$ (both move back)

NO MATCH:

Subsequence: $\max(dp[i-1][j], dp[i][j-1])$ (skip from X or Y)

Substring: 0 (RESET! contiguous)

LCS vs Knapsack Comparison

Aspect	0/1 Knapsack	LCS
What changes?	items (n), capacity	index i, index j
Choice	pick or skip item	match or skip char
On match/pick	$val + dp[i-1][w-wt]$	$1 + dp[i-1][j-1]$
On mismatch	$dp[i-1][w]$	$\max(dp[i-1][j], dp[i][j-1])$
Base case	$dp[0][w] = 0$	$dp[0][j] = dp[i][0] = 0$

All 16 Problems - Master Conversion Table

Problem	Formula using LCS	LC #
LCS length	$dp[m][n]$	1143
Longest Common Substring	$\max dp[i][j]$ (reset on mismatch)	-
Print LCS	backtrack through dp table	-
SCS length	$m + n - LCS(X, Y)$	-
Print SCS	backtrack, add non-LCS chars	-
Min Insert + Delete A->B	$(m - LCS) + (n - LCS)$	-
Longest Repeating Subseq	$LCS(s, s)$ with $i \neq j$	-

Is A subseq of B?	$LCS(A,B) == \text{len}(A)$	392
Count subsequences	dp + on match, carry on mismatch	115
Longest Palindrome Subseq	$LCS(s, \text{reverse}(s))$	516
Min del for palindrome	$n - LCS(s, \text{reverse}(s))$	-
Min ins for palindrome	$n - LCS(s, \text{reverse}(s))$	-
Longest Palindrome Substr	expand around center $O(n^2)$	5
Count Palindrome Substrs	expand around center $O(n^2)$	647
Edit Distance	$1 + \min(\text{insert}, \text{delete}, \text{replace})$	72
Interleaving Strings	$dp[i][j] = \text{OR}(\text{from } s1, \text{from } s2)$	97

Edit Distance (Levenshtein) - Key Formulas

```

MATCH:    dp[i][j] = dp[i-1][j-1]           # FREE! no operation needed
NO MATCH:  dp[i][j] = 1 + min(
            dp[i][j-1],                       # INSERT (add char to word1)
            dp[i-1][j],                       # DELETE (remove from word1)
            dp[i-1][j-1]                     # REPLACE (swap char)
        )
BASE:      dp[i][0] = i,  dp[0][j] = j       # delete/insert all chars

```

TRICK: Edit Distance vs LCS

If only INSERT and DELETE allowed (no REPLACE):

$\text{edit_distance} = m + n - 2 * \text{LCS}(\text{word1}, \text{word2})$

This reduces to the min-insertions-deletions problem!

5-Finger Trick for LCS Revision

THUMB: "MATCH = diagonal (i-1,j-1) + 1"

INDEX: "MISMATCH = max(up, left) for SUBSEQ, 0 for SUBSTR"

MIDDLE: "Palindrome = $LCS(s, \text{reverse}(s))$ "

RING: "SCS = $m + n - \text{LCS}$ "

PINKY: "Min ops = $(m - \text{LCS}) + (n - \text{LCS})$ "

Chapter 5: MCM / Partition DP Family

8 problems - split range $[i, j]$ at every k

Identification: How to spot MCM pattern

Is there a STRING or ARRAY? --> YES
 Do I need to SPLIT/PARTITION it at every possible k ? --> YES
 Does left part + right part give me the answer? --> YES = MCM!
 "partition", "split", "break", "merge" --> MCM pattern
 "parenthesize", "range $[i, j]$ " --> MCM pattern

The Universal MCM Template (memorize this!)

```
def solve(arr, i, j):
    # Step 1: BASE CASE
    if i >= j:
        return 0

    # Step 2: INITIALIZE
    ans = float('inf') # or float('-inf') for MAX problems

    # Step 3: TRY ALL PARTITIONS
    for k in range(i, j):
        # Step 4: LEFT + RIGHT + MERGE COST
        temp = solve(arr, i, k) + solve(arr, k+1, j) + cost(i, k, j)
        # Step 5: OPTIMIZE
        ans = min(ans, temp) # or max, or +=

    return ans
```

Bottom-Up: DIAGONAL Filling (NOT row-by-row!)

TRICK: The KEY difference from Knapsack

Knapsack fills ROW by ROW (item by item)
 MCM fills DIAGONALLY by GAP SIZE!

```
for gap in range(1, n): # outer: gap size
    for i in range(n - gap): # inner: start index
        j = i + gap
        dp[i][j] = ... # fill using smaller gaps
```

All 8 MCM Problems - Master Table

Problem	Base Case	k Range	Cost Formula	Optimize	LC #
---------	-----------	---------	--------------	----------	------

MCM	$i \geq j \rightarrow 0$	i to j-1	$a[i-1]*a[k]*a[j]$	MIN	-
Palindrome P.	isPalin $\rightarrow 0$	i to j-1	1 per cut	MIN	132
Boolean Par.	$i=j \rightarrow T/F$	operators	truth table	COUNT	GFG
Scramble Str	$s1==s2 \rightarrow T$	1 to n-1	swap/no-swap	OR	87
Egg Drop	$e=1 \rightarrow f$	1 to f	$1+\max(b,s)$	MIN	887
Burst Balloon	$i > j \rightarrow 0$	i to j	$a[i-1]*a[k]*a[j+1]$	MAX	312
Merge Stones	$i=j \rightarrow 0$	step K-1	rangeSum	MIN	1000
Polygon Tri.	$j-i < 2 \rightarrow 0$	i+1 to j-1	$v[i]*v[k]*v[j]$	MIN	1039

Burst Balloons - The Hardest Trick

TRICK: Think LAST to burst, NOT FIRST!

If we think 'burst first' $\rightarrow$ neighbors keep changing $\rightarrow$ impossible!

If we think 'burst LAST in range [i,j]' $\rightarrow$ neighbors are fixed: i-1 and j+1

Add dummy 1s: $nums = [1] + nums + [1]$

Cost of bursting k LAST: $nums[i-1] * nums[k] * nums[j+1]$

Egg Dropping - 4 Levels of Optimization

Level 1: Recursion $O(2^f)$ -- try every floor k

Level 2: Memoization $O(e*f^2)$ -- cache (eggs, floors)

Level 3: Memo + Binary Search $O(e*f*\log f)$ -- crossover point

Level 4: Inverted DP $O(e*\log f)$ -- "given trials, find max floors"

TRICK: break_cost increases, survive_cost decreases as k goes up

Boolean Parenthesization - Operator Truth Tables

AND (&): TrueWays = $LT * RT$
FalseWays = $LF*RF + LF*RT + LT*RF$

OR (|): TrueWays = $LT*RT + LT*RF + LF*RT$
FalseWays = $LF * RF$

XOR (^): TrueWays = $LT*RF + LF*RT$
FalseWays = $LT*RT + LF*RF$

Memory: AND = both true, OR = at least one true, XOR = exactly one true

Chapter 6: Standalone DP Problems

Important problems that don't fit the big 3 families

Problem 1: Count Ways to Reach Nth Step (Climbing Stairs) [Easy]

Companies: Amazon, Google, Microsoft, Meta, Apple

$dp[i] = dp[i-1] + dp[i-2]$ (Fibonacci!)

Base: $dp[0]=1$, $dp[1]=1$. Space $O(1)$: only need prev 2 values.

TRICK: Generalization

If steps allowed = $\{1,2,3\}$: $dp[i] = dp[i-1] + dp[i-2] + dp[i-3]$

If steps from set S : $dp[i] = \sum(dp[i-s] \text{ for } s \text{ in } S)$ = Coin Change!

Problem 2: Jump Game [Medium]

Companies: Amazon, Google, Microsoft, Meta

Given $nums[i]$ = max jump from position i , can you reach the end?

Greedy: track farthest reachable. If $i > \text{farthest}$ --> stuck!

```
def jump_game(nums):
    farthest = 0
    for i in range(len(nums)):
        if i > farthest: return False
        farthest = max(farthest, i + nums[i])
    return True
```

TRICK: DP exists but Greedy is $O(n)$ optimal

DP approach: $dp[i]$ = can reach index i ? $O(n^2)$

Greedy: track farthest position $O(n)$ -- ALWAYS present this!

Problem 3: Robbery / House Robber [Medium]

Companies: Amazon, Google, Microsoft, Meta, Goldman Sachs

Max money robbing non-adjacent houses.

$dp[i] = \max(nums[i] + dp[i-2], dp[i-1])$

(rob this + skip prev) (skip this)

```
def rob(nums):
    prev2, prev1 = 0, 0
    for num in nums:
        prev2, prev1 = prev1, max(num + prev2, prev1)
```

```
return prev1
```

TRICK: Follow-ups

House Robber II (circle): run twice on $[0..n-2]$ and $[1..n-1]$, take max

House Robber III (tree): DFS returning (rob, skip) pair per node

Problem 4: Knight's Tour on Phone Keypad [Medium]

Companies: Google, Amazon, Microsoft

Count distinct phone numbers of length n using knight moves on keypad.

Jump Map (memorize!):

0->[4,6] 1->[6,8] 2->[7,9] 3->[4,8] 4->[0,3,9]

5->[] 6->[0,1,7] 7->[2,6] 8->[1,3] 9->[2,4]

$dp[digit] = \sum(dp[prev] \text{ for } prev \text{ in moves}[digit])$

Only 10 states --> $O(1)$ space with rolling array!

Problem 5: Largest Square Submatrix With All 1s [Medium]

Companies: Amazon, Google, Microsoft, Goldman Sachs

$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ if cell='1'

The MIN is the bottleneck! A square needs all 3 neighbors to be large enough.

TRICK: Return AREA not side!

Answer = $\max_side * \max_side$

Problem 6: Word Wrap [Medium-Hard]

Companies: Google, Amazon, Microsoft

Arrange words on lines to minimize cost of extra spaces.

$dp[i] = \min(cost(i,j) + dp[j+1])$ for all valid j where words $i..j$ fit

TRICK: Key details

Cost = $(extra_spaces)^3$ (or 2 , ask interviewer!)

Last line cost = 0 (no penalty for trailing spaces)

Greedy packing is NOT optimal -- that's why we need DP!

Chapter 7: Ultimate Cheat Sheet

Everything on quick-reference pages - your last-minute revision

Pattern Recognition - See This? Think That!

You See This...	Think This!
Pick or skip + capacity/sum	0/1 Knapsack
Infinite supply / reuse items	Unbounded Knapsack
Two strings, what's common?	LCS
String vs its reverse	LCS -> Palindrome
Convert string A to B	Edit Distance (LCS variant)
Split/partition a range	MCM
Can't pick adjacent elements	House Robber
Count ways with 1/2 steps	Fibonacci DP
Largest square in matrix	Matrix DP: $\min(\text{top}, \text{left}, \text{diag}) + 1$
Min trials worst case	Egg Drop (MCM variant)
Boolean expression ways	Boolean Parenthesization (MCM)
Burst/merge elements	Burst Balloons (MCM, think LAST)

Formula Cheat Sheet (memorize these!)

LCS length	= $\text{dp}[m][n]$
Longest Palindrome	= $\text{LCS}(s, \text{reverse}(s))$
SCS length	= $m + n - \text{LCS}$
Min del for palindrome	= $n - \text{LCS}(s, \text{reverse}(s))$
Min insert + delete	= $(m - \text{LCS}) + (n - \text{LCS})$
Edit Distance	= $1 + \min(\text{insert}, \text{delete}, \text{replace})$
Equal Partition	= Subset Sum with target = $\text{totalSum} / 2$
Target Sum	= Count Subsets with sum = $(\text{total} + \text{target}) / 2$
Min Subset Diff	= $\text{total} - 2 * (\text{best S1} \leq \text{total}/2)$

Complexity Summary - All Families

Family	Recursion	Memo/BU	Space Opt	Key
0/1 Knapsack	$O(2^n)$	$O(n*W)$	$O(W)$ 1D	Right to Left
Unbounded KS	$O(2^n)$	$O(n*W)$	$O(W)$ 1D	Left to Right
LCS	$O(2^{(m+n)})$	$O(m*n)$	$O(\min(m,n))$	2 rows
MCM	$O(2^n)$	$O(n^3)$	$O(n^2)$	Diagonal fill
Edit Distance	$O(3^{(m+n)})$	$O(m*n)$	$O(n)$	2 rows
Egg Drop	$O(2^f)$	$O(e*f^2)$	$O(e)$	Inverted DP

The 3-Line Memo Conversion (works EVERYWHERE)

```
# 1. Create: memo = {} (or 2D array [-1])
# 2. Check: if key in memo: return memo[key]
# 3. Store: memo[key] = result; return memo[key]
```

Key Differences to Never Confuse

Aspect	0/1 Knapsack	Unbounded
Item reuse	At most ONCE	UNLIMITED
2D: include row	<code>dp[i-1][...]</code>	<code>dp[i][...]</code>
1D: loop direction	RIGHT to LEFT	LEFT to RIGHT
Keyword	'each item once'	'infinite supply'

Aspect	Knapsack (row)	MCM (diagonal)
Fill order	Row by row	Gap by gap
Depends on	Previous row only	Many smaller ranges
Space reduce?	YES: 1D array	NO: need full 2D
Typical time	$O(n*W)$	$O(n^3)$

Interview Flow (follow this EVERY time)

1. READ problem --> identify pattern (Knapsack? LCS? MCM? Other?)
2. DRAW choice diagram on whiteboard
3. WRITE recursion with base case
4. CONVERT to memoization (add 3 lines)
5. OFFER bottom-up if asked (show you know both)
6. OPTIMIZE space if asked (1D array trick)
7. STATE complexity: Time $O(?)$, Space $O(?)$

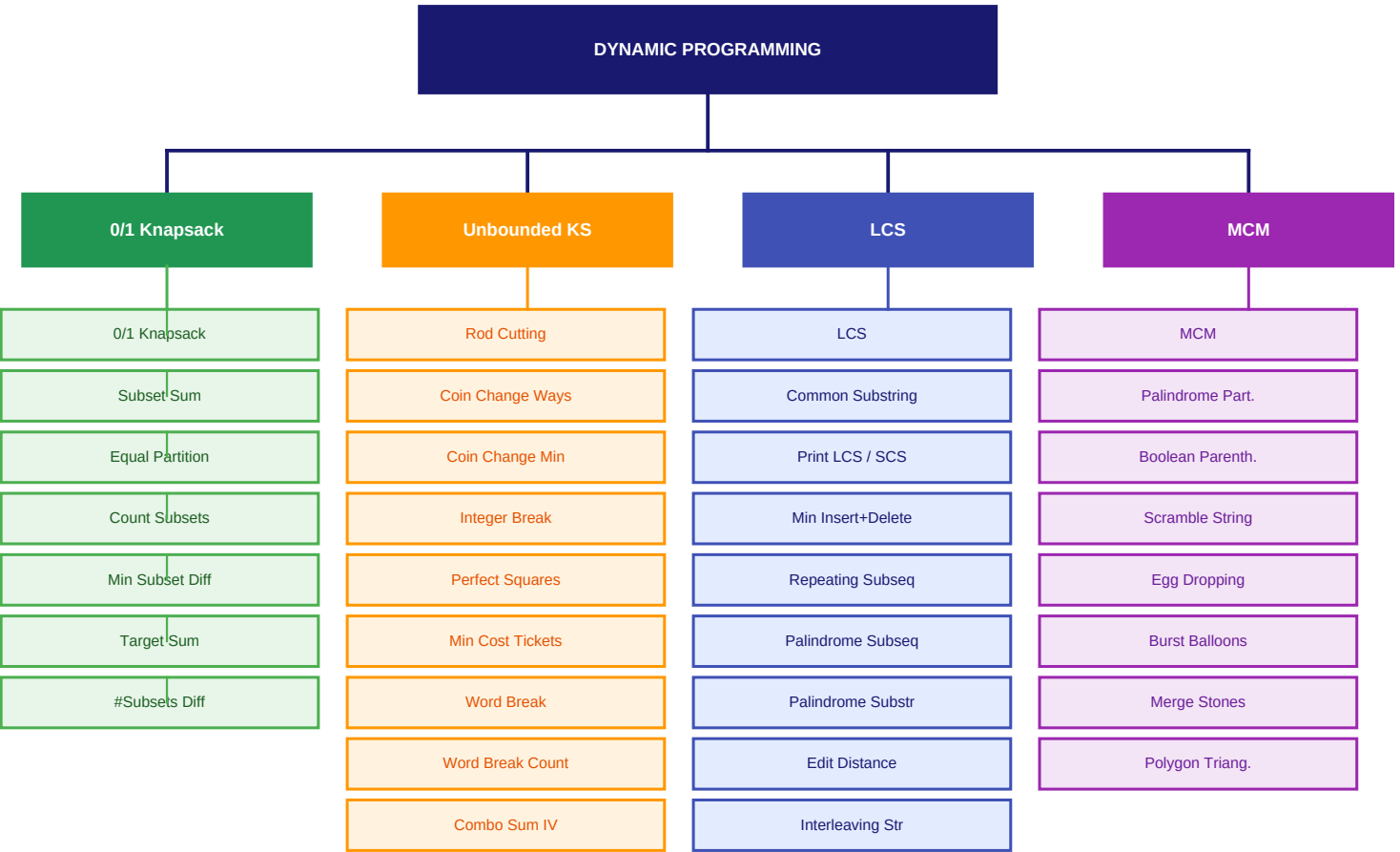
You've got this! Go crush those FAANG interviews!

Chapter 8: Visual Architecture Diagrams

Complete visual maps of all DP families, relationships, and conversion flows

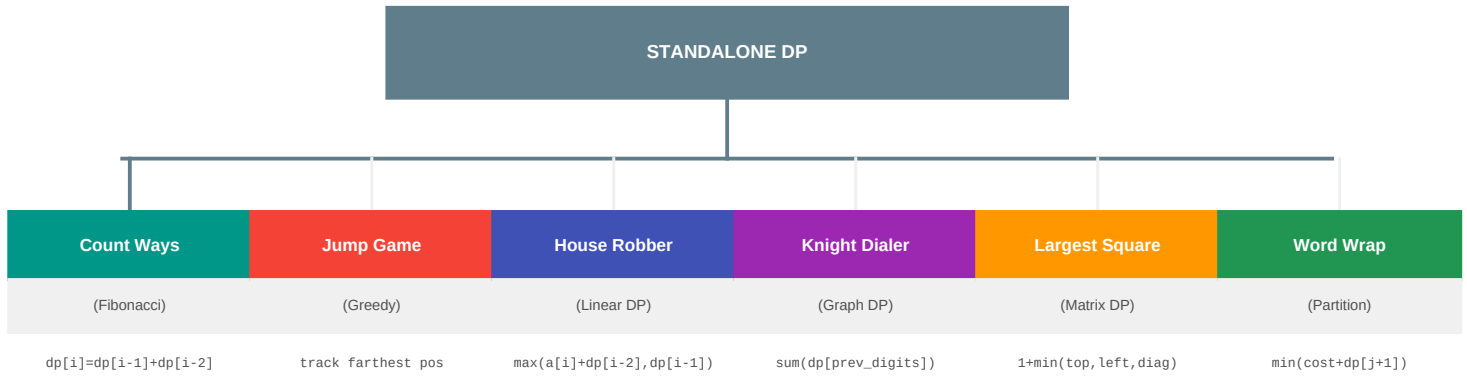
DP Family Architecture Diagram

Master map: 4 patterns, 41 problems, all from 4 templates



Total: 7 + 9 + 9 + 8 = 33 core problems (+8 standalone = 41 problems)

Standalone DP Problems Map

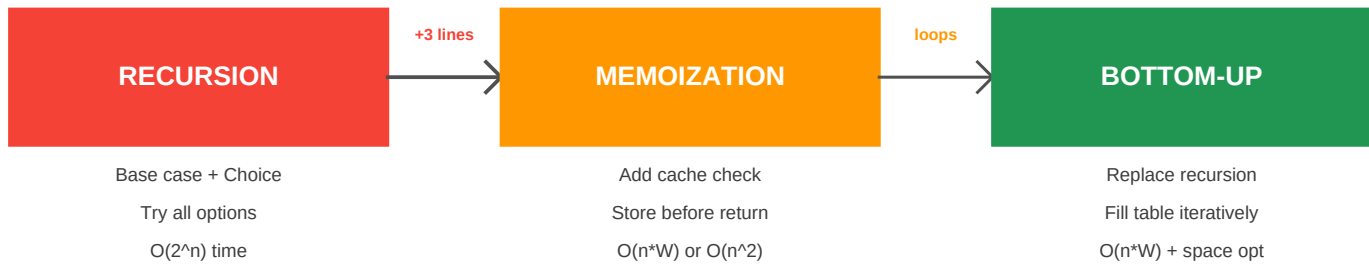


Each Problem Has 4 Approaches:



The DP Conversion Pipeline

How to convert any recursive solution to optimal DP in 3 steps



How Each Family Fills the DP Table

0/1 KNAPSACK	Row by row (i = items, j = capacity) j: RIGHT to LEFT in 1D $dp[i-1]$ = previous row
UNBOUNDED KS	Row by row (i = items, j = capacity) j: LEFT to RIGHT in 1D $dp[i]$ = SAME row (reuse!)
LCS	Row by row (i = string1, j = string2) Match: diagonal +1, Mismatch: $\max(\text{up}, \text{left})$ Space: 2 rows only
MCM	DIAGONAL by gap size (gap = j - i) Small gaps first, then larger Cannot reduce to 1D!

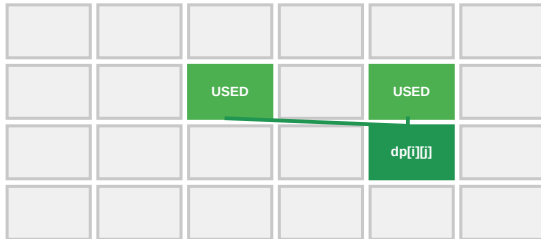
Conversion Rules (Recursion to Bottom-Up)

Base case	if $n==0$: return 0	-->	$dp[0][j] = 0$	(first row/col init)
Recursive call	$f(n-1, w)$	-->	$dp[i-1][j]$	(table lookup)
Return value	return $\max(a, b)$	-->	$dp[i][j] = \max(a, b)$	(store in cell)
Changing vars	(n, w)	-->	loop dimensions	(i, j)
Function call	$f(i, k) + f(k+1, j)$	-->	$dp[i][k] + dp[k+1][j]$	(MCM style)

0/1 vs Unbounded Knapsack: Visual Difference

0/1 KNAPSACK

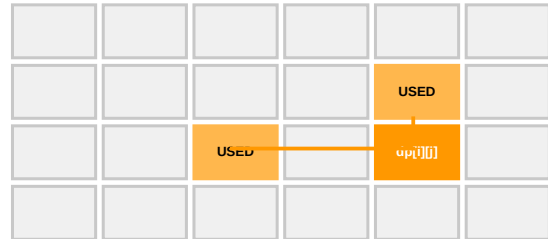
Uses $dp[i-1][j-wt]$ = PREVIOUS ROW



1D: j from W DOWN TO $wt[i]$
(RIGHT to LEFT)

UNBOUNDED KNAPSACK

Uses $dp[i][j-wt]$ = SAME ROW



1D: j from $wt[i]$ UP TO W
(LEFT to RIGHT)

The ONE Character Difference:

0/1: $val[i-1] + dp[i-1][j-wt[i-1]]$ $\leftarrow i-1$ (GO to prev row)

Unbounded: $val[i-1] + dp[i][j-wt[i-1]]$ $\leftarrow i$ (STAY on same row)

MCM: Diagonal Table Filling

Unlike Knapsack (row-by-row), MCM fills by GAP SIZE along diagonals

	j=0	j=1	j=2	j=3	j=4
i=0	1	6	10	13	15
i=1	-	2	7	11	14
i=2	-	-	3	8	12
i=3	-	-	-	4	9
i=4	-	-	-	-	5

Fill Order by Gap Size:

	Gap 0 (diagonal - base)
	Gap 1 (pairs)
	Gap 2 (triples)
	Gap 3 (quads)
	Gap 4 (full range = ANSWER)

The Diagonal Filling Loop:

for gap in range(1, n):	# gap = j - i
for i in range(n - gap):	# starting index
j = i + gap	
dp[i][j] = INF	
for k in range(i, j):	# try all split points
cost = dp[i][k] + dp[k+1][j] + merge_cost	
dp[i][j] = min(dp[i][j], cost)	